STUDENT CURRICULUM

Industry 4.0 IO-Link

CP7395 · IM4330 Training Kit

Field	Value
Course Code	CP7395
Title	Industry 4.0 IO-Link
Level	Engineering level — HNC/degree, engineers, system integrators
Total Time	10–12 hours
Worksheets	11
Prerequisites	CP3723 recommended — Worksheets 1–3 may run in parallel
Issue	2.0
Supplier	Matrix Technology Solutions Ltd



Course Overview

Course Code	CP7395
Title	Industry 4.0 IO-Link
Level	Engineering level — HNC/degree, engineers, system integrators
Total Time	10–12 hours
Worksheets	11
Prerequisites	CP3723 recommended — Worksheets 1–3 may run in parallel

What You Will Learn

By the end of this course you will be able to:

- Describe the full hardware and software architecture of an Industry 4.0 sensor system, including the edge-gateway pattern
- Trace the data path from IO-Link sensor through MQTT and the edge device to a browser-based HMI, and explain the HTTP/ISDU return path
- Decode raw IO-Link process data (PDin) hex values into engineering units using data types and scale factors, including two's complement for negative values
- Classify IO-Link data as process, service, or event data and explain where each type is routed and why
- Apply the NAMUR NE107 health-state symbols (Failure / Maintenance Required / Out of Specification / Function Check) to real sensor events
- Quantify the MTTR reduction from IO-Link and calculate the annual downtime and financial saving for a 40-sensor installation
- Map IO-Link data into PLC and OPC UA architectures, including the Y-Path concept, and identify the equivalent layers in this app
- Identify any connected sensor from its Vendor ID and Device ID without physical inspection, and use identity data to order the correct spare part
- Explain the PT100 resistance curve and IEC 60751 accuracy classes, and decode a temperature sensor PDin including negative and open-circuit values
- Apply calibration offset engineering to correct sensor drift via ISDU, and calculate Remaining Useful Life (RUL) from cycle-count data
- Decode and encode the Banner CL50 Pro 3-byte PDout command word from first principles, bit by bit

System Overview

Component	Model	Connection	Role
Sensors (Ports 1–4)	Proximity / Capacitive / Temperature / Light Stack	M12 3-wire to AL1350	Measure process data / receive PDout commands
IO-Link Master	IFM AL1350	192.168.7.4	Hub — all sensors plug in here, publishes MQTT
Edge Device	Raspberry Pi 4	192.168.7.2 (eth0) / DHCP (wlan0)	Runs backend, bridges IO-Link subnet to building LAN
HMI Screen	Browser dashboard	WebSocket from Pi	Live dashboard display

Data flows left to right (primary path): the AL1350 publishes sensor process data to Mosquitto via MQTT every 500 ms. The Raspberry Pi subscribes, decodes the PDin payloads, and pushes the result to connected browsers via WebSocket. HTTP polling runs in parallel for ISDU reads, device-tree information, and as a fallback.

Data flows right to left (ISDU): the browser sends parameter read/write requests through the FastAPI backend to the AL1350 HTTP service endpoint. ISDU commands travel from browser → WebSocket → FastAPI → HTTP → AL1350 → sensor.

Equipment on the Bench

Port	Device	Model	Technical Notes
1	Proximity Sensor	Omron E2E-X16MB1T12	IO-Link V1.1 COM3 (230.4 kbps). PDin: 2 bytes — byte 0 = monitor output (oscillation amplitude, uint8); byte 1 bit 0 = OUT1, bit 4 = instability alarm, bit 5 = over-approach alarm. Full ISDU access. Vendor ID 612, Device ID 131090.
2	Capacitive Sensor	RS Pro / Carlo Gavazzi M18	IO-Link 1.1. PDin bit 0 = output state; detection count accumulated in the backend from OFF→ON transitions. ISDU writable: SP1 (threshold, index 60/sub 1), teach commands.
3	Temperature Sensor	IFM TV7105	IO-Link 1.1. PDin: 4 bytes. Bytes 0–1 = signed int16 × 0.1 = °C. Bytes 2–3 carry SP1/SP2 flags. ±0.1 °C resolution. Open-circuit fault reads as −40.0 °C (FE70).
4	Light Stack	Banner CL50 Pro	PDout-only. 3-byte (24-bit) command word controls colour, animation, intensity, and speed. No PDin. Write via AL1350 `pdout/setdata` service call.
—	IO-Link Master	IFM AL1350	8 IO-Link/digital ports. IP: 192.168.7.4. Publishes MQTT every 500 ms; responds to HTTP service calls (ISDU).
—	Edge Device	Raspberry Pi 4	FastAPI + uvicorn on 127.0.0.1:8000. Mosquitto MQTT broker. eth0: 192.168.7.2 (IO-Link subnet). wlan0: building LAN (DHCP). Bridges the two isolated networks.

Network Topology

Layer	Address	Role
IFM AL1350	192.168.7.4	Publishes MQTT every 500 ms; serves HTTP for ISDU, device tree, supervision
Raspberry Pi	192.168.7.2 (eth0, static) / DHCP (wlan0)	Runs FastAPI, uvicorn, Mosquitto, decoder.py; bridges the IO-Link subnet to the building LAN
Browser (this app)	WebSocket /ws	Receives merged MQTT + HTTP data as JSON

WS1 — Your System: A Technical Brief

Estimated time	Dashboard	Prerequisites
15 minutes	Edge Device page, Admin / Connection Diagnostics	CP3723 recommended

Why This Matters

Every engineering project starts with a system brief. Before writing code or interpreting data you need to know what hardware is on the bench, how it is networked, and what each component's role is in the overall architecture. This worksheet walks through the complete hardware inventory, confirms everything is live, and tests your technical understanding of the system.

Learning Objectives

- Identify all components in the IM4330 kit by name, IP address, and technical role
- Describe the network topology — IO-Link subnet, edge gateway, building LAN
- Confirm system health using the Connection Diagnostics and Edge Device pages
- Calculate the WebSocket message rate implied by the AL1350's MQTT publish interval

Content — Architecture Overview

An SVG architecture diagram and the network topology block (above) walk through the full path: sensor → AL1350 → Mosquitto (MQTT, port 1883) → Raspberry Pi (FastAPI/uvicorn) → WebSocket → browser. A live system-status panel shows the master connection dot, connected port count, WebSocket message counter, and time since last update.

Concept — MQTT publish rate: the AL1350 publishes every 500 ms. Message rate = 1000 ms ÷ 500 ms = **2 messages per second**.

Concept — dual-NIC bridging: the Raspberry Pi has two network interfaces — eth0 (192.168.7.2, static, IO-Link subnet) and wlan0 (DHCP, building LAN). It bridges the two otherwise-isolated networks — the classic edge-gateway pattern.

Task — Hardware and Verification Checklist

Locate each component on the bench and tick it when confirmed:

- **IO-Link Master (IFM AL1350)** — IP **192.168.7.4**. All sensors connect here.
- **Port 1 — Omron E2E-X16MB1T12** — IO-Link V1.1 COM3 (230.4 kbps). Full ISDU access.
- **Port 2 — Capacitive Sensor** — SSC1 SP1 threshold + teach commands writable via ISDU.
- **Port 3 — IFM TV7105** — ±0.1 °C, PDin 4 bytes, bytes 0–1 = int16 ÷ 10.
- **Port 4 — Banner CL50 Pro** — PDout-only status light stack.
- **Edge Gateway (Raspberry Pi 4)** — eth0 192.168.7.2 (static) / wlan0 (DHCP).
- **This dashboard** — receives merged data over WebSocket; never talks directly to the AL1350.

Then confirm live system health:

- Circuit breaker on Connection Diagnostics is **CLOSED**
- Latency graph on Connection Diagnostics is live (not flat-lined)
- Backend service is **Online** on the Edge Device page
- WebSocket message counter is incrementing at approximately **2 messages per second**

Knowledge Check

Q1. What is the IP address of the IFM AL1350 IO-Link Master?

A. 192.168.7.4

B. 192.168.7.2 — this is the Raspberry Pi's eth0 address

C. 192.168.1.1

Q2. The AL1350 publishes every 500 ms. Approximately how many WebSocket messages should arrive at the browser each second?

A. 20 messages per second

B. 2 messages per second — 1000 ÷ 500 = 2

C. 500 messages per second

Q3. Why does the Raspberry Pi have two network interfaces (eth0 and wlan0)?

A. It allows two IO-Link masters to be connected simultaneously

B. It doubles bandwidth by bonding eth0 and wlan0 together

C. It bridges the isolated IO-Link subnet (192.168.7.x) to the building LAN — the Pi is the edge gateway

Q4. According to the hardware table, how many IO-Link ports have a sensor connected?

A. 4 — Ports 1 through 4 each have a device connected

B. 8 — every port on the AL1350 is populated

C. 2

WS2 — How This App Works

Estimated time	Dashboard	Prerequisites
20 minutes	Edge Device page, Admin / Connection Diagnostics	CP3723 recommended

Why This Matters

Understanding the full data path — from IO-Link sensor through the edge device to your browser — is the foundation of any Industry 4.0 integration. The same architecture pattern (sensor → MQTT broker → edge gateway → WebSocket → HMI) is used in industrial IoT platforms at scale.

Learning Objectives

- Explain why MQTT is preferred over HTTP polling for 500 ms sensor data
- Describe the Pi's role as an edge gateway bridging two isolated networks
- Trace the data path: AL1350 → Mosquitto → FastAPI → decoder.py → WebSocket → browser
- Understand how the backend handles AL1350 power cycles and maintains its MQTT subscription
- Interpret circuit breaker state and round-trip latency

Background — Software Layers

Each layer is explained with an analogy:

Layer	Role	Analogy
Mosquitto (MQTT broker)	Receives the AL1350's push and fans it out	A WhatsApp group — publish once, every subscriber gets it
FastAPI + uvicorn	Backend API server	A bilingual receptionist — translates between HTTP/ISDU and WebSocket/JSON
`decoder.py`	Parses raw PDin hex into engineering values	A barcode scanner — reads the code, returns the meaning
WebSocket `/ws`	Persistent push channel to the browser	A phone call left open — no need to redial each time
HMI dashboard	Renders the decoded data	A car instrument panel

Why MQTT over HTTP polling? MQTT uses publish/subscribe — the AL1350 pushes data the moment it is ready, without the Pi having to request each cycle. This is lower overhead than repeated HTTP GET requests every 500 ms.

The subscription-loss problem: the AL1350 loses its MQTT subscriptions when it power-cycles. The backend calls `ensure_mqtt_subscription()` on every startup to re-register the push subscription automatically — a technician who power-cycles the AL1350 does not need to also restart the Pi.

Circuit breaker and latency: the HTTP fallback path is protected by a circuit breaker — CLOSED (healthy) normally, OPEN after 5 consecutive failures, HALF-OPEN after a 15 s recovery timeout. Round-trip latency is normally 20–80 ms; consistently above ~200 ms indicates AL1350 load or IO-Link subnet congestion, not a Wi-Fi problem on the browser side.

Task — Trace the Data Path

1. Watch the WebSocket message counter climb by roughly 20 in 10 seconds (≈2/s).

2. Check the circuit breaker and latency panel on Connection Diagnostics.
3. Warm the TV7105 probe with your hand on the Dashboard and watch the live decode chain: °C value → int16 → raw hex, all updating together in real time.

Knowledge Check

Q1. Why is MQTT better suited than HTTP polling for delivering sensor data every 500 ms?

A. MQTT uses publish/subscribe — the AL1350 pushes data the moment it is ready, without the Pi having to request each cycle

B. HTTP is push-based and MQTT requires a new TCP request each cycle

C. MQTT guarantees delivery in all network conditions by queuing until acknowledged

Q2. The AL1350 loses its MQTT subscriptions when it power-cycles. How does the backend handle this?

A. It waits for the user to manually re-subscribe via Settings

B. It switches permanently to HTTP polling after any restart

C. It calls `ensure_mqtt_subscription()` on every backend startup, re-registering the push subscription automatically

Q3. If the FastAPI backend service on the Pi crashed, what would you expect to observe?

A. Live data would stop and the WebSocket would disconnect — the browser shows offline

B. The WebSocket message counter would continue to increment normally

C. The AL1350 would automatically restart the backend via a built-in watchdog

Q4. What does a consistently high round-trip latency value on Connection Diagnostics indicate?

A. The AL1350 is under load or there is congestion on the IO-Link subnet

B. The browser's chart rendering cannot keep up

C. The Pi's Wi-Fi interface is saturated by other building-network traffic

WS3 — How Each Sensor Works (Protocol Level)

Estimated time	Dashboard	Prerequisites
20 minutes	Active Port Details (Raw PDin)	Complete WS1

Why This Matters

Every IO-Link sensor sends a raw binary process data (PDin) payload. The master delivers it as a hex string; the backend's `decoder.py` parses each device type differently based on its IODD specification. Understanding this decode layer is essential for integrating sensors into PLCs, historians, and digital twin platforms — and for fault-finding when a raw value looks wrong.

Learning Objectives

- Describe the PDin decode pipeline: raw hex → `decoder.py` → engineering value
- Identify the bit fields in the proximity sensor PDin (OUT1, instability alarm, over-approach alarm)
- Explain what the capacitive detection count represents and why it is useful
- Decode a temperature sensor PDin hex value using int16 and scale ×0.1
- Explain why the CL50 PDout requires more care to build than a simple PDin decode

Background — PDin Byte Layout Per Device

Port 1 — Proximity Sensor (Omron E2E-X16MB1T12) — PDin is 2 bytes:

Byte	Bit	Field	Meaning
Byte 0	—	Monitor output	Oscillation amplitude, uint8 (~80 in free air)
Byte 1	Bit 0	OUT1	1 = object detected
Byte 1	Bit 4	Instability Alarm	1 = target near edge of sensing range
Byte 1	Bit 5	Over-Approach Alarm	1 = target too close to sensor face
Byte 1	Bit 6	Warning flag	
Byte 1	Bit 7	Error flag	

Port 2 — Capacitive Sensor — bytes 0–1 carry field strength; byte 2 bit 0 is the SSC1 switching output. The backend accumulates a session detection count, incremented on each OFF→ON transition.

Port 3 — Temperature Sensor (IFM TV7105) — signed int16, scale ×0.1: `0x00F5` (245) → 24.5 °C; `0x02EE` (750) → 75.0 °C; `0xFFCE` (−50) → −5.0 °C.

Port 4 — Light Stack (Banner CL50 Pro) — PDout only, no PDin; this worksheet includes an interactive builder (colour swatches, animation picker, live hex preview, "Send to light" button).

Task — Live PDin Panels

Work through each port's live decode panel: Port 1 proximity PDin decode, Port 2 dielectric panel, Port 3 temperature decode panel, and Port 4's interactive PDout builder.

Knowledge Check

Q1. Byte 1 of the proximity sensor PDin reads `0x01`. What does this mean?

A. OUT1 is active — an object is detected

B. The sensor is reporting an error

C. The monitor output amplitude is 1

Q2. Why is the capacitive sensor's running detection count useful?

A. It replaces the need for a switching output entirely

B. It confirms the ISDU connection is alive

C. A running count lets you track fills/parts over a shift without watching the screen continuously

Q3. A TV7105 PDin decodes to `0x02EE`. What temperature does this represent?

A. 2.99 °C

B. 7.5 °C

C. 75.0 °C — 0x02EE = 750 decimal × 0.1

Q4. Why does building a correct CL50 PDout value require more care than decoding a simple sensor PDin?

A. PDout packs multiple unrelated fields (colour, animation, intensity, speed, audible) into specific bit ranges across 3 bytes — getting one field wrong can change several others at once

B. PDout uses a different hex encoding to PDin

C. The CL50 requires a checksum byte that other devices don't

WS4 — Benefits for Maintenance

Estimated time	Dashboard	Prerequisites
15 minutes	Port status and events	Complete WS1–WS2

Why This Matters

Mapping IO-Link features directly to maintenance problems is how you build the business case for Industry 4.0 investment to a manager who does not care about protocols — only about downtime and cost.

Learning Objectives

- Match specific IO-Link data features (events, identity, parameter storage, cycle counts) to the maintenance problem each one solves
- Interpret the MTTR comparison between standard wiring and IO-Link
- Calculate an annual time saving from a fleet-wide MTTR reduction

Background — Four Features, Four Problems

Feature	Maintenance Problem Solved	Kit Example
Diagnostic events	"Which sensor failed and why?" without an on-site visit	Port 1 instability alarm
Device identity	"What is the exact spare part number?"	Port 1 Vendor 612, Device 131090
Parameter storage / ADR	"Configuration is lost after a swap"	TV7105 SP1/RP1/SP2/RP2/calibration offset auto-restore
Operating hours / cycle count	"When should this be replaced, before it fails?"	Port 2 capacitive ISDU count

MTTR comparison:

Step	Standard wiring	IO-Link
Walk to sensor	10 min	(alert removes the walk-to-diagnose step)
Identify fault	8 min	1 min (alert)
Replace / reconfigure	12 min	2 min (order) + 4 min (swap)
Verify	5 min	1 min
Total	**~35 min**	**~8 min**

Saving per fault: **≈27 minutes**. For 40 sensors at 1 fault/sensor/year: 40 × 27 min = 1,080 min = **18 hours saved per year**.

Task — Matching Exercise and Repair-Workflow Challenge

Matching exercise — match each problem to the feature that solves it:

Problem	Feature
Sensor fails without warning	Diagnostic events

Replacing a sensor loses configuration	Parameter storage (ADR)
Unknown which sensor failed	Diagnostic events
Planning when to replace	Diagnostic events (trend from event history)

5-step live repair-workflow scenario simulating a Port 2 failure:

4. **Alert** — set the CL50 to Red/Flashing via the PDout builder
5. **Identify** — read the Port 2 device identity
6. **ADR** — read SP1 (index 60/sub 1)
7. **Cycle count** — trigger Port 2 by hand and watch the live counter
8. **Sign off** — restore the CL50 to Green/Steady

> **NOTE** Steps 1 and 3 (alert + parameter recovery) account for 22 of the 27 minutes saved per fault.

WS5 — IODD Files & Decoding IO-Link Data

Estimated time	Prerequisites
20 minutes	Complete WS1–WS4

Why This Matters

The IODD (IO Device Description) is the specification that defines how to decode a device's raw PDin and ISDU data. Without it, raw hex is meaningless — this worksheet takes you from a blank hex string to a confident manual decode.

Learning Objectives

- Explain what an IODD file is and where to find one for an unknown device (`ioddfinder.io`)
- Describe how the master auto-identifies a device and how the backend translates that into a decode table (`backend/device_parameters.py`)
- Convert a raw hex value to a signed or unsigned integer and apply a scale factor
- Apply the two's-complement rule for negative int16 values

Background — From Hex to Engineering Value

At power-up, the AL1350 reads the connected sensor's Vendor ID and Device ID and loads its IODD. In this app, `backend/device_parameters.py` is the hard-coded translation table built from those IODD definitions.

Example IODD extract — TV7105 SP1: Index 583, Subindex 0, datatype INT16, scale ×0.1, raw range −600 to 1500.

Decode steps:

9. Two hex bytes → one 16-bit integer (e.g. `01 F4` → 500 decimal)
10. If the datatype is signed and the value is ≥ 32768, apply two's complement: `signed = unsigned − 65536` (e.g. `0xFF9C` = 65436 → 65436 − 65536 = **−100** → ×0.1 = **−10.0 °C**)
11. Multiply by the scale factor to get the engineering value

Data types on this kit: uint8 (proximity monitor output), uint16 (capacitive SP1 depending on device), int16 (TV7105 SP1/RP1/SP2/RP2/calibration offset), int32 (Contrinex photoelectric SP1 — not on this kit's live ports but referenced in the registry).

Worked examples:

- TV7105 SP1 `0118` → 280 → **28.0 °C**
- Calibration offset `FFD0` → −48 → **−4.8 °C**
- RP1 `00E6` → 230 → **23.0 °C**
- Capacitive SP1 `0064` → **100 counts**

Task — Live ISDU Decode Verification

Perform three live ISDU reads and confirm your manual hex decode matches the value shown on screen:

12. TV7105 SP1 — index 583, sub 0
13. Capacitive SP1 — index 60, sub 1
14. Calibration offset — index 681, sub 0

Knowledge Check

Q1. You need the ISDU parameter map for a device not yet in this app's registry. Where do you find it?

- **A.** `ioddfinder.io` — the official IODD repository
- **B.** The AL1350's operating manual
- **C.** The device's physical label

WS6 — Diagnostics Data & NAMUR NE107 Health States

Estimated time	Dashboard	Prerequisites
15 minutes	Port Details, Events	Complete WS1–WS3

Why This Matters

IO-Link carries three distinct kinds of data, each routed and used differently. NAMUR NE107 is the cross-vendor standard that turns raw fault codes into a small set of universally-understood health symbols — the same symbols appear on HMIs from every major automation vendor.

Learning Objectives

- Classify a given piece of sensor data as process, service, or event data
- Explain the latency and routing difference between the three data types
- Map real sensor events on this kit to the four NAMUR NE107 symbols
- Interpret what the WebSocket actually carries

Background — Three Data Channels

Type	Trigger	Latency	Example
Process data (PDin)	Cyclic, every scan	500 ms	Live temperature value
Service data (ISDU)	Acyclic, on request	10–200 ms	Reading a filter-time parameter
Event data	Triggered by a condition	Immediate	Instability alarm code + port + timestamp, feeds a CMMS

NAMUR NE107 — four health symbols:

Symbol	Meaning	Kit Example
✗ Failure	Device cannot function	Connection lost
△ Maintenance Required	Will fail soon if not addressed	Instability alarm
⊙ Out of Specification	Operating outside rated conditions	Over-approach alarm; SP1 exceeded
⌞ Function Check	Temporary state during a config/teach operation	Calibration write in progress

Task — Classification Exercise and Live Tiles

Classification matching — classify each item as process, service, or event data:

Item	Type
Temperature value	Process
Device replacement	Event
Filter-time parameter read	Service
Object-present flag	Process
Short circuit fault	Event

Then observe 3 live tiles showing process, service, and event data behaving differently, and tick off the sign-off checklist.

Knowledge Check

Q1. What does the WebSocket connection to this dashboard carry?

A. Process and event data merged into one stream

B. Only raw PDin, unprocessed

C. Service data only

Q2. The proximity sensor's instability alarm fires. Which NE107 symbol applies?

A. ✗ Failure

B. △ Maintenance Required — the target is near the edge of range and will cause intermittent detection if not corrected

C. ⊙ Out of Specification

Q3. Why does NAMUR NE107 matter for a maintenance team working across multiple sensor vendors?

A. It reduces sensor cost

B. It is a legal requirement for IO-Link certification

C. It gives every vendor's HMI the same four symbols for health state, so an operator does not need to learn a different fault-code scheme per manufacturer

WS7 — PLC, HMI & OPC UA Integration

Estimated time	Dashboard	Prerequisites
15 minutes	Port Status, Simulate Fault	Complete WS1–WS4

Why This Matters

This worksheet is what distinguishes an installer from a systems integrator: understanding how IO-Link data reaches a PLC scan cycle, an OPC UA server, and ultimately the cloud — and where this app's own architecture sits within that picture.

Learning Objectives

- Describe how PDin data reaches a PLC's process image
- Compare this app's ~500 ms WebSocket update rate to fieldbus cyclic rates (PROFINET/EtherNet-IP, 1–10 ms)
- Explain the role of OPC UA (IEC 62541) as a vendor-neutral information model
- Describe the Y-Path concept and identify which branch this app's architecture resembles

Background

PDin → PLC scan cycle (4 steps): sensor → IO-Link master maps PDin into its process image → fieldbus (PROFINET/EtherNet-IP) cyclically exchanges that process image with the PLC → the PLC's user program reads the mapped tag on its own scan cycle.

Fieldbus vs this app: PROFINET/EtherNet-IP typically cycle every 1–10 ms — far faster than this app's ~500 ms WebSocket update, because a PLC control loop needs much tighter timing than a monitoring HMI.

OPC UA (IEC 62541): a vendor-neutral companion specification that maps a device's Vendor ID, Device ID, and PDin fields onto standard OPC UA information-model nodes, letting IT-side software (MES, historians, cloud) consume the same data without vendor-specific drivers.

The Y-Path: cyclic process data continues to flow to the PLC as normal (the control path). Acyclic data — serial numbers, operating hours, error codes — is sent directly from the IO-Link master's OPC UA server to IT systems, bypassing the PLC entirely (the IT-facing branch). This app's architecture — Pi reads ISDU data directly from the AL1350 over HTTP and serves it to a browser, independent of any PLC — structurally resembles the Y-Path's IT-facing branch.

Task — Live Verification

Part A: warm the TV7105 by hand and watch the cyclic (PDin) value rise smoothly — this is the process-data path.

Part B: trigger an ISDU read and watch the CL50 flash blue then return to green — visible proof of the acyclic (service-data) path completing.

Knowledge Check

Q1. How does PDin data reach a PLC's scan cycle?

A. The sensor writes directly into PLC memory over IO-Link

B. The PLC polls each sensor individually over its own network connection

C. The IO-Link master maps PDin into its process image, which the fieldbus cyclically exchanges with the PLC's own process image

Q2. What is the key advantage of a web-based HMI (like this dashboard) over hard-wired panel indicators?

A. It is always cheaper to install

B. It requires no network infrastructure

C. A single dashboard can display live process data, historical trends, and remote parameter access — without dedicated wiring per indicator

Q3. What does a latency spike on Connection Diagnostics most likely indicate about this app's architecture?

A. AL1350 load or IO-Link subnet congestion — not a PLC scan-cycle issue, since this app is not in the PLC control path

B. A PLC scan overrun

C. A fieldbus cable fault

Q4. Which OPC UA data path does this app's architecture most closely resemble?

A. The pure cyclic control path through the PLC

B. The Y-Path's IT-facing branch — acyclic data delivered directly from the master to IT/browser without passing through a PLC

C. A dedicated SCADA historian connection

WS8 — Case Study: MTTR, TCO & Predictive Maintenance

Estimated time	Dashboard	Prerequisites
20 minutes	Simulate Fault, Active Port Details	Complete WS1–WS5

Why This Matters

This worksheet quantifies the reactive-vs-predictive maintenance economics using this kit's own cycle-count, event, and drift data — the exact case a maintenance engineer has to make to a finance department.

Learning Objectives

- Calculate annual downtime hours from an MTTR figure and fault frequency across a sensor fleet
- Convert a downtime saving into an annual financial saving
- Compare reactive, planned-preventive, and predictive maintenance strategies by cost and risk
- Calculate Remaining Useful Life (RUL) from cycle-count data and a rated lifetime

Background — Scenario: 40 Sensors, 1 Fault/Sensor/Year

MTTR breakdown:

Step	Standard	IO-Link
Walk to sensor	10 min	0 min (alert)
Power down	5 min	—
Reconfigure/replace	12 min	4 min (swap) + 2 min (order)
Verify	8 min	2 min
Total	**35 min**	**8 min**

Annual downtime: Standard = 35 × 40 ÷ 60 = **23.3 h/year**. IO-Link = 8 × 40 ÷ 60 = **5.3 h/year**. **Saving: 18 h/year.**

Financial impact at £5,000/hour production value: Standard = **£116,500/year**. IO-Link = **£26,500/year**. **Saving: £90,000/year.**

Three maintenance strategies:

Strategy	MTTR	Cost/fault	Risk
Reactive	35 min	~£2,917	High
Planned preventive	8 min	~£200	Medium
Predictive	8 min	~£200	Low

RUL worked example: capacitive sensor has done 7.5M of its rated 10M cycles at 1,000 cycles/day. Cycles remaining to 90% threshold (9M) = 1.5M ÷ 1,000/day = **1,500 days ≈ 4.1 years**. This lets a CMMS work order be raised for a planned replacement window instead of an emergency callout.

CMMS integration example: an event (e.g. fault code `0x8C11`, port, timestamp) can auto-generate a CMMS work order — this is the event-data channel from WS6 in practice.

Task — Calculation Table

Work through the downtime and financial calculation table yourself, then check against the figures above. A scale-up extension task asks you to repeat the calculation for a fleet larger than 40 sensors.

Knowledge Check

Q1. Using the scenario figures, what is the annual downtime saving from switching to IO-Link?

- **A.** 18 hours per year — 23.3 h − 5.3 h
- **B.** 5.3 hours per year
- **C.** 23.3 hours per year

Q2. At £5,000/hour production value, what is the annual financial saving?

- **A.** £26,500
- **B.** £116,500
- **C.** £90,000 — 18 h × £5,000

Q3. What are the main drivers of lower Total Cost of Ownership (TCO) for IO-Link sensors?

- **A.** Lower unit purchase price
- **B.** Automatic parameter restore after replacement, plus remote diagnostics that avoid unnecessary site visits
- **C.** Longer warranty periods

Q4. What data does a CMMS integration typically consume from this system?

- **A.** Event data — fault codes, port, and timestamp
- **B.** Raw PDin only
- **C.** ISDU parameter values only

Q5. Using the RUL worked example, when should the Port 2 capacitive sensor be scheduled for replacement?

- **A.** Immediately — it has already exceeded its rated life
- **B.** In approximately 1,500 days (4.1 years) — schedule replacement before reaching 90% (9M cycles)
- **C.** There is no way to estimate this from cycle-count data

Q6. Which data sources feed a predictive-maintenance strategy on this kit?

- **A.** Cycle count only
- **B.** Cycle count, event frequency, and calibration/parameter drift together
- **C.** Live PDin value alone

WS9 — Device Identity: Vendor ID, Device ID & PDin

Estimated time	Dashboard	Prerequisites
15 minutes	IO-Link Master page — Port Status cards	CP3723 Chapter 8 recommended

Why This Matters

Labels wear off, records go missing, and sensors get swapped between machines. The Vendor ID and Device ID broadcast automatically by every IO-Link sensor at power-up let you confirm exactly what hardware is connected — and order the correct spare — directly from the dashboard, with no physical inspection required.

Learning Objectives

- Look up a Vendor ID against the known registry for this kit
- Combine Vendor ID and Device ID to identify an exact sensor model
- Use device identity to select the correct PDin decode table
- Use identity data to respond correctly to a live alarm condition

Background — Identity Registry

Known Vendor IDs on this kit: 310 = ifm, 342 = Contrinex, 612 = OMRON, 896/1586 = RS Pro (OEM branding of Carlo Gavazzi).

Device ID combinations on this kit:

Vendor ID	Device ID	Model
612	131090	OMRON E2E-X16MB1T12
310	733	IFM TV7105
1586	1052673	RS Pro capacitive (Carlo Gavazzi OEM, part 2377240)

The OMRON byte-order trap: the IODD counts `bitOffset` from the LSB of the 16-bit record — byte 0 and byte 1 are not in the intuitive order. Byte 0 is the monitor-output amplitude (not a status byte); byte 1 carries OUT1 and the alarm flags. A PDin of `5100` in free air is byte 0 = `0x51` (monitor ≈ 81) and byte 1 = `0x00` (all flags clear) — misreading byte 0 as flags produces a permanent false "instability alarm."

Task — Identity-Only Diagnosis

15. **Mystery sensor challenge (Port 4):** using only the identity fields — no physical inspection — determine which device is connected: (a) CL50 Pro, (b) a second OMRON sensor, or (c) TV7105.
16. **Manual PDin decode-by-hand check (Port 3).**
17. **Port 1 alarm live confirmation** — move a metal target near and away from the sensor face and confirm the instability alarm behaves as expected.

Knowledge Check

Q1. What Vendor ID does Port 1 report?

A. 612

B. 310

C. 342

Q2. Vendor ID 612 corresponds to which manufacturer?

A. ifm

B. OMRON

C. Contrinex

Q3. Why does the Device ID matter specifically when ordering a spare part, beyond just the Vendor ID?

A. It doesn't — Vendor ID alone is always sufficient

B. It sets the IP address of the replacement

C. Multiple products from the same vendor share a Vendor ID — the Device ID uniquely identifies the exact model, so the correct part number can be derived without a label or manual

Q4. Port 1's instability alarm (bit 4) is active. What is the correct maintenance response?

A. Check the target alignment and sensing distance — the target is near the edge of the sensing range

B. Replace the sensor immediately

C. Ignore it — instability alarms are informational only

Q5. A sensor reports Vendor ID 1586, Device ID 1052673. What should you order as a replacement?

A. An OMRON proximity sensor

B. A TV7105 temperature sensor

C. An RS Pro capacitive sensor (Carlo Gavazzi OEM)

WS10 — PT100 Temperature Sensors: From Resistance to Process Data

Estimated time	Dashboard	Prerequisites
25 minutes	CP3723 Worksheet 5	CP3723 Worksheet 5 recommended

Why This Matters

Understanding the PT100 element's underlying physics and how the TV7105 encodes it into PDin lets you decode a hex value without a datasheet in hand — and engineer a calibration offset correctly rather than guessing.

Learning Objectives

- Apply the PT100 resistance-temperature relationship
- Explain IEC 60751 accuracy classes and the TV7105's class
- Decode temperature PDin including negative values (two's complement) and the open-circuit default
- Engineer a calibration offset write to correct a known drift

Background — PT100 Physics

Resistance curve (IEC 60751, α = 0.00385/°C): R(T) = 100 × (1 + 0.00385 × T)

Temperature	Resistance
−40 °C	84.6 Ω
0 °C	100.0 Ω
25 °C	109.6 Ω
60 °C	123.1 Ω
100 °C	138.5 Ω
150 °C	157.8 Ω

Fault behaviour: an open circuit reads as an implausibly low resistance and defaults to **−40.0 °C** (`0xFE70`); a short circuit reads as an implausibly high resistance/temperature.

IEC 60751 accuracy classes: AA (tightest), A, and **B** — the TV7105's class. Each class defines a tolerance formula and resulting tolerance band at 0 °C, 25 °C, and 100 °C.

PDin encoding: int16 × 0.1, big-endian, bytes 0–1; bytes 2–3 carry SP1/SP2 flags.

Two's complement (signed int16): `signed = unsigned − 65536` when unsigned ≥ 32768.

Hex	Unsigned	Signed	°C
`00F5`	245	245	+24.5 °C
`FF9C`	65436	−100	−10.0 °C
`FE70`	65136	−400	−40.0 °C (open-circuit default)

Calibration offset engineering (index 681, ±10 °C): if a reference thermometer reads 50.0 °C and the sensor reads 53.0 °C, the sensor is 3.0 °C high — write **−3.0 °C** (`0xFFE2`) to correct it.

Setpoint/reset-point notes: SP1/SP2/RP1/RP2 are stored in the sensor's EEPROM and survive power cycles. The sensor enforces SP > RP for the same output.

Task — Live PT100 Verification

18. Practice decode exercises: `012C` → 30.0 °C; `0196` → 40.6 °C; `FF9C` → −10.0 °C; `FE70` → −40.0 °C (open-circuit).
19. Watch the live PT100 panel (raw hex → bytes → int16 → °C → computed resistance) update in real time.
20. Hand-encode a value and check it against the panel.
21. Perform live ISDU reads of the calibration offset (index 681) and SP1 (index 583) and compare against the live temperature.

WS11 — CL50 Pro: PDout Encoding Deep-Dive

Estimated time	Dashboard	Prerequisites
20 minutes	CP3723 Worksheet 6	CP3723 Worksheet 6 required

Why This Matters

Once you can read and write the CL50's 24 bits directly — without relying on the app's quick-colour buttons — you can control any PDout device, including from PLC ladder logic or structured text.

Learning Objectives

- Decode any 24-bit CL50 PDout hex value by hand, bit by bit
- Encode a PDout value from a written specification (colour, animation, speed, intensity)
- Explain the known firmware limitation with the Intensity Sweep animation mode

Background — Full PDout Bit Map

Byte	Bits	Field	Values
Octet 0	[7:6]	Audible	0=Off, 1=On, 2=Pulsed, 3=SOS
Octet 0	[5:3]	Colour 2 Intensity (C2I)	0=High, 1=Low, 2=Medium, 3=Off
Octet 0	[2:0]	Colour 1 Intensity (C1I)	0=High, 1=Low, 2=Medium, 3=Off
Octet 1	[7:6]	Speed	0=Medium, 1=Fast, 2=Slow
Octet 1	[5:3]	Pulse Pattern	0=Normal, 1=Strobe, 2=Three Pulse, 3=SOS, 4=Random
Octet 1	[2:0]	Animation	0=Off, 1=Steady, 2=Flash, 3=Two Colour Flash, 4=Intensity Sweep
Octet 2	[7:4]	Colour 2 index	0–15
Octet 2	[3:0]	Colour 1 index	0–15

Colour index: 0=Green, 1=Red, 2=Orange, 3=Amber, 4=Yellow, 5=Lime, 6=Spring Green, 7=Cyan, 8=Sky Blue, 9=Blue, 10=Violet, 11=Magenta, 12=Rose, 13=White, 14=Custom1, 15=Custom2.

Worked decode example: `184203` — Octet 0 `0x18` = C1I 0 (High), C2I 3 (Off) → this is actually the AL1350 default pattern for Amber Flashing at High intensity when combined with Octet 1 `0x42` (Animation=Flash, Pulse Pattern=Normal) and Octet 2 `0x03` (Colour 1 = Amber). Result: **Amber, Flashing, Fast, High intensity.**

Firmware v1.0.2 limitation: Animation = 4 (Intensity Sweep) does not execute — the AL1350 accepts the write but the light shows a static colour instead of cycling. The software workaround is a backend breathing-loop that steps C1I through Low→Medium→High→Medium→Low using Animation = Steady.

Task — Decode, Encode, and Live Verification

Decode exercises: `000100`, `004312`, `198205`.

Encode exercises:

Specification	Correct Hex
Blue · Steady · Medium · High	`000109`
Cyan/Magenta · Two Colour Flash · Fast · High	`0043B7`

5-step live verification: write and decode real CL50 PDout hex via the Dashboard quick-colour buttons and the LED Config modal — including confirming the Animation = 4 firmware bug in person.